Cafenea — Teorie

Alex · adnotările Spring pe etape · câte o carte pe adnotare

Etapa 1 · Componente și bean-uri

`@SpringBootApplication` 1

Clasa de pornire: scanează pachetul ei cu subpachetele și pornește auto-configurarea.

Unde: `CafeneaApplication`, direct în `app`

```
@SpringBootApplication
public class CafeneaApplication {}
```

⚠ Găsește doar ce e în `app.*`; `config` rămâne nescanat.

`@ComponentScan` 1

Spune în ce pachete caută Spring componente.

Unde: inclus în `@SpringBootApplication`; nu-l scrii

```
@ComponentScan("app")
```

⚠ Face deja parte din `@SpringBootApplication`: nu-l pui a doua oară.

`@Component` 1

Spring creează un singur obiect (bean) și îl dă unde e cerut.

Unde: pe clasă; dependențele vin prin constructor

```
@Component
public class ViewCafenea {}
```

⚠ Fără `getINSTANCE()` și fără `new`: clasa neadnotată nu e bean.

`@Configuration` 1

Clasă în care declari bean-uri de mână.

Unde: `app.config.AppConfig`, `config.ConsolaConfig`

```
@Configuration
public class AppConfig {}
```

⚠ `ConsolaConfig` e în afara lui `app`: scanarea n-o vede.

`@Bean` 1

Metodă al cărei rezultat devine bean; pentru clase pe care nu le adnotezi.

Unde: într-o `@Configuration`

```
@Bean
Scanner scanner() {
    return new Scanner(System.in);
}
```

⚠ Numele bean-ului = numele metodei.

`@Import` 1

Aduce o `@Configuration` pe care scanarea n-o vede.

Unde: pe `CafeneaApplication`

```
@Import(ConsolaConfig.class)
```

⚠ Fără el lipsește `Scanner`, iar aplicația nu pornește.

Etapa 2 · Roluri

`@Service` 2

`@Component` pentru stratul de logică.

Unde: pe cele 2 `*ServiceImpl`

```
@Service
public class ProdusServiceImpl {}
```

⚠ Apare la `getBeanNamesForAnnotation(Service.class)`; cu `@Component` lista iese goală.

`@Repository` 2

`@Component` pentru stratul de date.

Unde: pe cele 2 `*RepositoryImpl`

```
@Repository
public class ProdusRepositoryImpl {}
```

⚠ Apare la `getBeanNamesForAnnotation(Repository.class)`.

Etapa 3 · Configurare din fișier

`@Value` 3

Injectează o valoare din configurare.

Unde: parametru de constructor

```
CalculatorPret(
  @Value("${cafenea.tva}") double tva)
```

⚠ Cheia lipsă: aplicația nu pornește.

`@ConfigurationProperties` 3

Leagă proprietățile cu un prefix de câmpurile unui obiect.

Unde: `record CafeneaProperties` din `config`

```
@ConfigurationProperties(
  prefix = "cafenea")
```

⚠ Singură nu face bean: îi trebuie `@EnableConfigurationProperties`.

`@EnableConfigurationProperties` 3

Face bean dintr-o clasă `@ConfigurationProperties`.

Unde: pe `AppConfig`

```
@EnableConfigurationProperties(
  CafeneaProperties.class)
```

⚠ Necesar fiindcă `config` nu e scanat.

`@PropertySource` 3

Încarcă un fișier `.properties` în plus.

Unde: pe `AppConfig`

```
@PropertySource(
  "classpath:program.properties")
```

⚠ Citește doar `.properties`, nu YAML.

Etapa 4 · Mai multe bean-uri de același tip

`@Autowired` 4

Alege constructorul pe care îl folosește Spring.

Unde: pe constructorul complet din `ViewCafenea`

```
@Autowired
public ViewCafenea(..., BonPrinter detaliat)
```

⚠ Cu 2 constructori și fără el, Spring caută unul fără parametri.

`@Primary` 4

Bean-ul ales implicit dintre mai mulți de același tip.

Unde: pe `SimpluBonPrinter`

```
@Component("simplu")
@Primary
```

⚠ Un singur `@Primary` per tip.

`@Qualifier` 4

Cere un bean anume, după nume.

Unde: pe parametrul din `ViewCafenea`

```
@Qualifier("detaliat")
BonPrinter detaliat
```

⚠ Numele trebuie să fie cel din `@Component("detaliat")`.

`@Order` 4

Ordinea într-o listă: mai mic = primul.

Unde: pe cele 2 reduceri, în `List<Reducere>`

```
@Order(1)
public class ReducereMeniu
```

⚠ Ordinea schimbă totalul: 1-2 dă 37.93, invers 37.81.

Etapa 5 · Câte obiecte și când se creează

`@Scope` 5

`singleton` = unul pentru toți; `prototype` = unul nou la fiecare cerere.

Unde: `Comanda` (prototype), `ComandaRepositoryImpl` (singleton)

```
@Scope("prototype")
ObjectProvider<Comanda> p;
p.getObject();
```

⚠ Un prototype injectat direct într-un singleton se creează o singură dată.

`@Lazy` 5

Bean creat la prima folosire, nu la pornire.

Unde: pe `RaportVanzari` și pe parametrul din `ViewCafenea`

```
@Component
@Lazy
public class RaportVanzari
```

⚠ Dacă lipsește de pe parametru, se creează tot la pornire.

`@DependsOn` 5

Creează alt bean înainte de acesta.

Unde: pe `ProdusRepositoryImpl`

```
@DependsOn("jurnalCafenea")
```

⚠ Doar ordinea creării; nu injectează nimic.

Etapa 6 · Ciclul de viață

`@PostConstruct` 6

Rulează o dată, după injectare.

Unde: `incarca()` din `ProdusRepositoryImpl`

```
@PostConstruct
void incarca() {...}
```

⚠ Constructorul nu mai adaugă produse.

`@PreDestroy` 6

Rulează o dată, la oprire.

Unde: `salveaza()` din `ComandaRepositoryImpl`

```
@PreDestroy
void salveaza() {...}
```

⚠ Rulează doar dacă contextul se închide.

`@Bean(initMethod, destroyMethod)` 6

La fel, pentru clase pe care nu le poți adnota.

Unde: `CasaDeMarcat` în `AppConfig`

```
@Bean(initMethod = "deschide",
      destroyMethod = "inchide")
```

⚠ Numele metodelor sunt șiruri: o greșeală apare abia la pornire.

Etapa 7 · Condiții și profiluri

`@Profile` 7

Bean doar cu un profil activ; `!` = profil inactiv.

Unde: `MeniuTestRepository`, `ProdusRepositoryImpl`

```
@Profile("dev")
@Profile("!dev")
```

⚠ În fiecare profil trebuie să rămână un singur `ProdusRepository`.

`@ConditionalOnProperty` 7

Bean doar dacă proprietatea are valoarea dată.

Unde: pe cele 2 reduceri

```
@ConditionalOnProperty(
  name = "cafenea.reduceri.activ",
  havingValue = "true")
```

⚠ Fără proprietate, bean-ul lipsește.

`@ConditionalOnMissingBean` 7

Bean doar dacă lipsește altul de același tip.

Unde: `@Bean`-ul `ConsolaNotificator`

```
@Bean
@ConditionalOnMissingBean(
  Notificator.class)
```

⚠ Se retrage când apare un alt `Notificator`.

Etapa 8 · Evenimente

`@EventListener` 8

Rulează la evenimentul din parametru; serviciul nu știe de el.

Unde: `Bucatarie`, `Statistici`, `JurnalCafenea`

```
@EventListener
void pregateste(
  ComandaFinalizataEvent e)
```

⚠ Pe `ApplicationStartedEvent` merge; `ApplicationReadyEvent` vine după meniu.

Etapa 9 · Fire și programare în timp

`@EnableAsync` 9

Pornește firele pentru `@Async`.

Unde: pe `AppConfig`

```
@EnableAsync
```

⚠ Fără ea, `@Async` e ignorat și meniul stă 2 s.

`@Async` 9

Rulează pe alt fir; apelantul nu așteaptă.

Unde: ascultătorul din `Bucatarie`, firul `task-N`

```
@Async
@EventListener
void pregateste(...)
```

⚠ Contoarele atinse din mai multe fire: `AtomicInteger`.

`@EnableScheduling` 9

Pornește planificatorul.

Unde: pe `AppConfig`

```
@EnableScheduling
```

⚠ Fără ea, `@Scheduled` nu rulează niciodată.

`@Scheduled` 9

Rulează periodic, la un interval.

Unde: `raport()` din `Statistici`

```
@Scheduled(fixedRateString =
  "${cafenea.raport.ms}")
```

⚠ Intervalul vine din configurare, ca șir.